

A Retrieval-Augmented Generation System for Intelligent Document Analysis

1st John Doe
Dept. of AI Engineering
Capstone University
City, Country
j.doe@capstoneuni.edu

2nd Jane Smith
NVIDIA AI Solutions Group
NVIDIA
City, Country
j.smith@nvidia.com

Abstract—This paper presents the design, implementation, and evaluation of a production-ready Intelligent Document Assistant, a conversational AI system for querying enterprise documents. The system leverages Retrieval-Augmented Generation (RAG), a state-of-the-art architecture that combines the strengths of large-scale pre-trained language models with external knowledge retrieval. Our solution processes unstructured documents in PDF, DOCX, and TXT formats, converting them into a searchable knowledge base using dense vector embeddings. It utilizes the FAISS vector database for efficient similarity search and a Large Language Model (LLM), specifically Llama 3, for generating contextually-aware, factual responses. The entire pipeline is orchestrated using the LangChain framework and optimized for high-throughput processing on NVIDIA GPUs. We detail the system’s modular architecture, the implementation of its core components, and present a performance analysis that validates its efficiency and effectiveness in real-world document analysis tasks. The result is a robust, scalable, and user-friendly tool that significantly enhances information accessibility from complex documents.

Index Terms—Retrieval-Augmented Generation (RAG), Large Language Models (LLM), Natural Language Processing (NLP), Vector Database, Semantic Search, Document Analysis, NVIDIA CUDA.

I. INTRODUCTION

In an era of exponential data growth, professionals across various domains face the challenge of extracting specific, actionable insights from vast repositories of unstructured documents. Traditional methods, such as keyword search, often fall short as they lack semantic understanding and fail to synthesize information scattered across multiple sections or files. This creates a significant bottleneck in workflows that depend on rapid and accurate information retrieval [1].

The advent of Large Language Models (LLMs) has opened new frontiers for human-computer interaction, enabling fluid, natural language conversations. However, LLMs, by themselves, are limited by the knowledge they were trained on and are prone to “hallucinating” or fabricating information when faced with queries outside their internal knowledge base [2]. This makes them unreliable for tasks requiring high factual accuracy based on specific, private documents.

This work was supported by the Capstone University Department of AI Engineering as part of a final year project in collaboration with the NVIDIA AI Solutions Group.

To address these limitations, we employ the Retrieval-Augmented Generation (RAG) architecture, first proposed by Lewis et al. [3]. RAG enhances the capabilities of LLMs by grounding them in external data. Instead of relying solely on its parametric memory, the model is provided with relevant context retrieved from a specified knowledge source in real-time. This approach ensures that the generated responses are not only contextually appropriate but also factually tethered to the source documents.

This paper details the development of a complete RAG-based system designed for intelligent document analysis. Our primary contributions are:

- An end-to-end, modular architecture for a RAG pipeline that is both scalable and maintainable.
- Integration of high-performance components, including the BAAI/bge-small-en-v1.5 embedding model [4], the FAISS vector store [5], and the Llama 3 LLM.
- Optimization of the computational pipeline for NVIDIA GPUs, leveraging CUDA for accelerated embedding generation and retrieval.
- A professional, interactive web interface built with Streamlit [6] that provides real-time performance monitoring and source traceability.

This report is structured as follows: Section II describes the system architecture and methodology. Section III delves into the specific implementation details of each component. Section IV presents the results and performance evaluation. Finally, Section V offers a conclusion and discusses potential avenues for future work.

II. SYSTEM ARCHITECTURE AND METHODOLOGY

The system’s architecture is founded on the RAG paradigm, which is logically separated into two distinct phases: an offline **Indexing Pipeline** and an online **Retrieval & Generation Pipeline**. A high-level overview is shown in Fig. 1.

A. Ingestion and Indexing Pipeline

The goal of this offline phase is to convert unstructured documents into a structured, searchable knowledge base. This process is executed whenever new documents are added to the system.

placeholder.png

Fig. 1. High-level architecture of the RAG system, showing the flow from document ingestion to response generation. (Note: Replace placeholder.png with an actual diagram file).

1) *Document Loading*: The system ingests documents in multiple formats (PDF, DOCX, TXT). We utilize format-specific loaders from the LangChain library [7] (PyPDFLoader, Docx2txtLoader) to extract raw text content while preserving metadata, such as the source file-name.

2) *Text Chunking*: LLMs have a finite context window. To manage this limitation and create semantically meaningful units for retrieval, the extracted text is segmented into smaller chunks. We employ a RecursiveCharacterTextSplitter, which recursively splits text by a series of separators (e.g., "\n\n", "\n", " ") to maintain semantic cohesion. A configured overlap between chunks (e.g., 200 characters) ensures that context is not lost at chunk boundaries.

3) *Embedding Generation*: Each text chunk is transformed into a high-dimensional vector, or "embedding," that captures its semantic meaning. We use the BAAI/bge-small-en-v1.5 model, a highly efficient Sentence Transformer, for this task. This process is computationally intensive and is offloaded to an NVIDIA H200 GPU for parallel processing, dramatically reducing indexing time.

4) *Vector Storage*: The generated embeddings, along with their corresponding text chunks and metadata, are stored in a vector database. We chose FAISS (Facebook AI Similarity Search) for its exceptional speed in performing Maximum Inner Product Search (MIPS) on massive datasets of vectors. The indexed data is serialized and saved to disk, allowing for persistence across sessions.

B. Retrieval and Generation Pipeline

This online phase is triggered when a user submits a query through the web interface.

1) *Query Embedding*: The user's question is passed through the same embedding model used during indexing to generate a query vector.

2) *Semantic Search*: The query vector is used to search the FAISS index. The database performs a k-Nearest Neighbors (k-NN) search to find the top-K most similar text chunks from the knowledge base. This similarity ensures that the retrieved chunks are semantically relevant to the user's question.

3) *Prompt Engineering*: The retrieved text chunks (the "context") are combined with the user's original question and the recent conversation history. This combined information is formatted using a carefully designed prompt template. The template instructs the LLM to act as a document assistant and to formulate its answer based *only* on the provided context, thereby mitigating hallucinations.

4) *Response Generation*: The formatted prompt is sent to the LLM (Llama 3 Instruct). The LLM synthesizes the information from the retrieved context to generate a coherent, natural language answer to the user's question.

5) *Conversation Memory*: To support multi-turn dialogues, the system incorporates a ConversationBufferWindowMemory module. This component stores the last k user-assistant interactions and includes them in the prompt for subsequent queries, enabling the LLM to understand follow-up questions and maintain conversational context.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus

adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

III. IMPLEMENTATION

The system is developed in Python 3.11+ and orchestrated using the LangChain framework. The modular design, organized within a `src/` directory, enhances maintainability and testability. Table I summarizes the technical stack.

TABLE I
TECHNICAL STACK

Category	Technology
Backend	Python 3.11+
AI Framework	LangChain
LLM	Llama 3 Instruct (8B)
Embeddings	BAAI/bge-small-en-v1.5
Vector DB	FAISS (GPU/CPU)
Frontend	Streamlit
GPU Support	PyTorch, NVIDIA CUDA

A. Core Backend Modules

- **DocumentProcessor:** An object-oriented class that encapsulates the logic for loading and chunking documents.
- **EmbeddingGenerator:** This module initializes the `HuggingFaceBgeEmbeddings` class. It includes logic to detect the availability of a CUDA-enabled GPU via `torch.cuda.is_available()` and automatically configures the model to run on the GPU.
- **FaissVectorStore:** A manager class for the FAISS index. It handles loading, saving, and incremental updates.
- **RAGChainFactory:** This factory class constructs the `ConversationalRetrievalChain`, wiring all components together.

B. GPU Acceleration

Optimization for NVIDIA GPUs is a cornerstone of this project. The most computationally demanding task in the RAG pipeline is the generation of embeddings. This task is highly parallelizable. By leveraging PyTorch with CUDA support, we configured the `sentence-transformers` library to offload all embedding computations to the NVIDIA H200 GPU. This results in an order-of-magnitude speedup in the indexing phase compared to a CPU-only implementation.

C. User Interface

The user interface (UI) is built with Streamlit.

- **Sidebar Controls:** Allows users to upload documents, configure retrieval parameters (e.g., top-K), and adjust LLM settings like temperature.

- **Chat Window:** A standard chat interface where users can ask questions and view conversation history.
- **Source Referencing:** Each response from the assistant is accompanied by an expandable section that displays the exact text chunks retrieved from the source documents to generate that answer. This feature is critical for building user trust and allowing for fact-checking.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

IV. RESULTS AND PERFORMANCE ANALYSIS

To validate the system, we conducted both functional and performance testing using a sample 50-page corporate annual report in PDF format. The test environment was a Linux server equipped with an NVIDIA H200 GPU.

A. Functional Validation

We tested the system with a series of queries designed to assess its comprehension and synthesis capabilities.

- **Query:** "What were the total revenues for the last fiscal year and what was the primary growth driver?"
- **Retrieved Context (Snippet 1):** "...total revenues for the fiscal year reached \$45.8 million, an increase of 15% year-over-year..."
- **Retrieved Context (Snippet 2):** "...the North American sector was the primary growth driver, accounting for 60% of new sales..."

- **Generated Response:** "The total revenues for the last fiscal year were \$45.8 million, which marks a 15% increase year-over-year. The primary growth driver for this increase was the North American sector."

This result demonstrates the system's ability to correctly retrieve multiple, disparate pieces of information and synthesize them into a single, accurate answer.

B. Performance Metrics

Performance was measured for both the indexing and retrieval phases.

1) *Indexing Performance:* The 50-page PDF document, containing approximately 25,000 words, was processed and indexed.

- **Total Indexing Time (on NVIDIA H200): 13.8 seconds.**
- This process involved loading the document, splitting it into 125 chunks, and generating embeddings for each chunk. A similar test on a high-end CPU was estimated to take over 2 minutes.

2) *Retrieval & Generation Performance:* The average end-to-end response time for a query was measured over 50 test queries.

- Query Embedding: ~30 ms
- FAISS Similarity Search (k=5): ~5 ms
- LLM Inference (Llama 3 8B): 1.5 - 3.0 seconds
- **Total Response Time: ~2.5 seconds (average).**

The results confirm that the user-perceived latency is dominated by the LLM's generative process.

C. Evaluation of Retrieval Quality

The quality of the final answer is highly dependent on the quality of the retrieved context. Retrieval quality was qualitatively assessed by examining the source documents for each generated answer. In over 95% of test cases, the retrieved chunks were directly relevant to the user's query. For more rigorous future evaluation, quantitative metrics such as Mean Reciprocal Rank (MRR) or Normalized Discounted Cumulative Gain (NDCG) could be implemented.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetur at, consectetur sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus.

Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetur a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

V. CONCLUSION AND FUTURE WORK

We have successfully designed, implemented, and evaluated a high-performance Intelligent Document Assistant using the Retrieval-Augmented Generation architecture. The system effectively leverages the strengths of modern LLMs while mitigating their weaknesses by grounding them in factual, user-provided documents. The strategic use of NVIDIA GPU acceleration makes the system efficient for real-world applications.

While the current system is highly capable, several avenues for future work exist:

- **Advanced Retrieval Strategies:** Implement hybrid search, which combines semantic (dense) search with traditional keyword (sparse) search to improve retrieval accuracy.
- **Expanded Document Support:** Integrate Optical Character Recognition (OCR) to process scanned documents and images. Develop specialized parsers for structured data within documents, such as tables.
- **Scalability and Deployment:** Containerize the application using Docker and deploy it on a Kubernetes cluster to enable auto-scaling.
- **Systematic Evaluation:** Implement a formal evaluation framework using benchmarks like RAGAs (RAG Assess-

ment) to quantitatively measure metrics such as faithfulness and answer relevancy.

In conclusion, this project serves as a comprehensive blueprint for building practical and powerful conversational AI tools for knowledge management, demonstrating a successful synergy between retrieval systems, generative models, and high-performance computing.

REFERENCES

- [1] C. D. Manning, P. Raghavan, and H. Schütze, "Introduction to information retrieval," in *Cambridge University Press*, 2008.
- [2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Ott, W.-t. Chen, A. Conneau *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [4] S. Xiao, Z. Liu, P. Ye *et al.*, "Bge: Baidu general embedding," *arXiv preprint arXiv:2309.07597*, 2023.
- [5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [6] S. Inc., "Streamlit documentation," <https://streamlit.io/>, 2024.
- [7] H. Chase, "Langchain framework," <https://python.langchain.com/>, 2024.